



2.- CAPÍTULO II: Programación Orientada a Objetos

Herencia

Marzo 2026 - Agosto 2026

Ciencia de Datos & Inteligencia Artificial

1. Herencia

1.1 Herencia en POO

1.2 Herencia en Python

1.3 Tipos de Herencia

1. Herencia

1.1 Herencia en POO

1.2 Herencia en Python

1.3 Tipos de Herencia

Herencia

La herencia permite crear nuevas clases a partir de clases ya existentes, reutilizando atributos y métodos. Gracias a este mecanismo, se evita la duplicación de código y se facilita el mantenimiento de los sistemas.

La herencia permite que una clase adquiera las características y comportamientos de otra clase.

La herencia es uno de los pilares fundamentales de la Programación Orientada a Objetos junto con la encapsulación, el polimorfismo y la abstracción.

En la vida real muchos objetos comparten características comunes.

Por ejemplo:

- Un automóvil tiene marca, modelo y velocidad.
- Una motocicleta tiene marca, modelo y velocidad.
- Un camión tiene marca, modelo y velocidad.

Si programamos cada clase por separado, estaríamos repitiendo código.

La herencia permite reutilizar atributos y métodos de una clase existente.

Supongamos que estamos desarrollando un sistema universitario.

Estudiante
nombre
apellido
cedula
correo
...

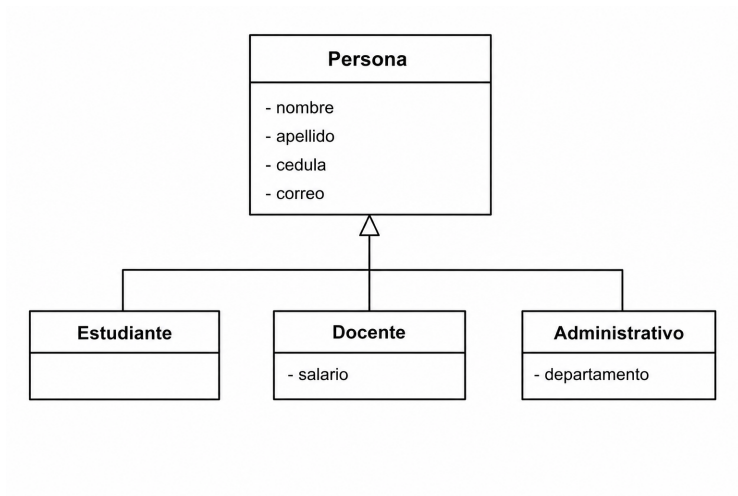
Docente
nombre
apellido
cedula
correo
salario
...

Administrativo
nombre
apellido
cedula
correo
departamento
...

Observamos que las tres clases poseen información repetida.

Si cada clase almacena los mismos atributos, se genera: Duplicación de código y Mayor dificultad de mantenimiento.

Solución integrado



La herencia es el mecanismo mediante el cual una clase puede adquirir atributos y métodos de otra clase. Permite crear nuevas clases a partir de clases ya existentes.

La clase que proporciona las características se denomina:

- Clase Padre
- Superclase
- Clase Base

La clase que recibe las características se denomina:

- Clase Hija
- Subclase
- Clase Derivada

La nueva clase hereda atributos y métodos de la clase original.

Clase Padre (Superclase)

Es la clase que contiene los atributos y comportamientos generales.

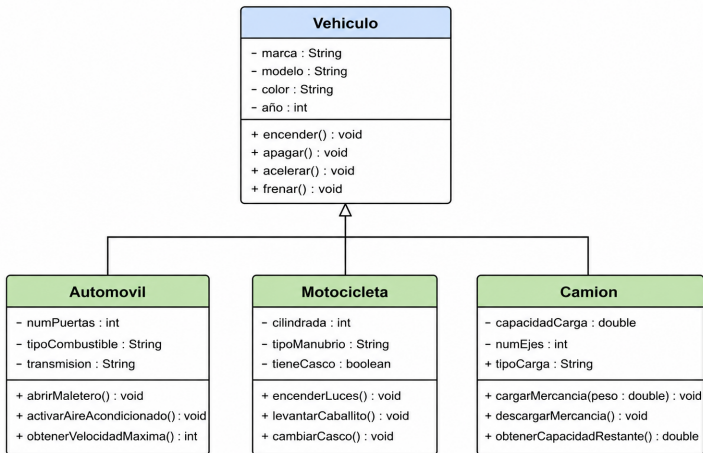
Clase Hija (Subclase)

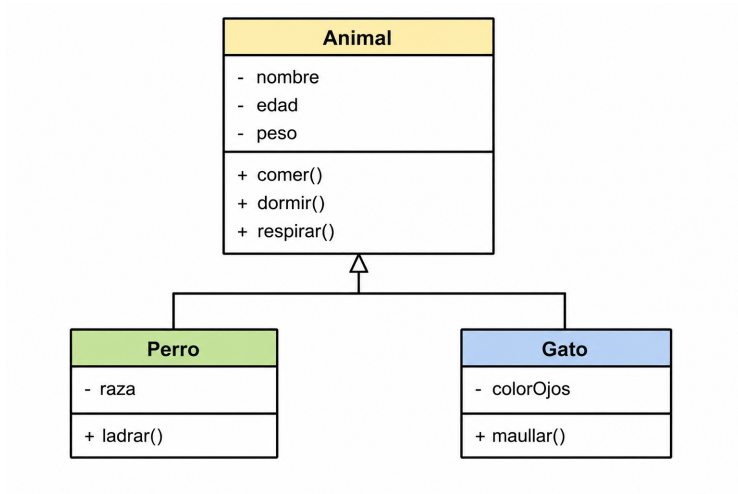
Es la clase que hereda las características de la clase padre y puede agregar nuevas funcionalidades.

Clase Padre: Es la clase general que contiene características comunes.

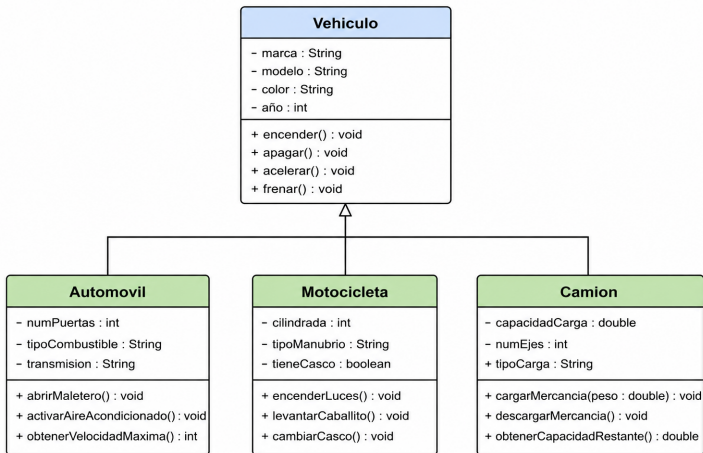
Vehiculo
marca modelo color salario
encender () apagar ()

Clase Hija: Es una clase especializada que hereda características de otra.





Clase Hija: Es una clase especializada que hereda características de otra.



Ventajas

- Reutilización de código.
- Menor mantenimiento.
- Mayor organización.
- Facilita la escalabilidad.

La herencia representa una relación *Es un*:

- Perro **ES UN** Animal
- Gato **ES UN** Animal
- Camión **ES UN** Vehículo
- Profesor **ES UNA** Persona
- Estudiante **ES UNA** Persona

1. Herencia

1.1 Herencia en POO

1.2 Herencia en Python

1.3 Tipos de Herencia

1.2 Definición

```
class Padre:  
    pass  
  
class Hija(Padre):  
    pass
```

```
class Persona:

    def __init__(self,nombre):
        self.nombre = nombre

    def saludar(self):
        print("Hola, soy", self.nombre)

class Estudiante(Persona):
    pass

e1 = Estudiante("Juan")
e1.saludar()
```

El atributo nombre y el método saludar() es heredado.

super(): Permite acceder a métodos de la clase padre. Generalmente se utiliza para invocar el constructor.

```
class Persona:

    def __init__(self,nombre):
        self.nombre = nombre

class Estudiante(Persona):

    def __init__(self,nombre,carrera):

        super().__init__(nombre)
        self.carrera = carrera
```

1. Herencia

1.1 Herencia en POO

1.2 Herencia en Python

1.3 Tipos de Herencia

Una clase hija hereda de una sola clase padre.

```
class Animal:  
    pass  
  
class Perro(Animal):  
    pass
```

Una clase hereda de otra clase derivada.

```
class Animal:  
    pass  
  
class Mamifero(Animal):  
    pass  
  
class Perro(Mamifero):  
    pass
```

Una clase hereda de varias clases.

```
class Volador:

    def volar(self):
        print("Volando")

class Nadador:

    def nadar(self):
        print("Nadando")

class Pato(Volador, Nadador):
    pass
```